ADR 001: Technology Stack for Mobile & AR Rice Disease/Pest Detection

Status: Proposed
Date: 2026-02-02
Author: Research Team (AI & CV)
Context: Rice disease and pest detection using lightweight ML/CV models (e.g., MobileNet/YOLO variants) deployed fully offline on entry-level Android devices and augmented reality (AR) glasses.[1] [2] [3]

1. Context and Problem Statement

Rice disease and pest detection requires real-time inference in the field, often with intermittent or no connectivity, on low-cost phones (2–4 GB RAM, mid-range Snapdragon/MediaTek) and AR glasses.[2]
Recent literature shows that lightweight CNNs (MobileNetV2/V3, ShuffleNet) and compact YOLO variants can reach >90–95% accuracy on rice leaves/pests while remaining deployable on mobile hardware.[3] [4] [5] [6] [7] [8] [2]
The architecture must cover: model development and training, optimization (compression, quantization), and deployment on Android and AR devices with offline operation and low latency (<100 ms per frame).[5] [2]

2. Decision

Adopt the following **technologies, languages, frameworks and libraries** as the standard stack:

1. **Model Development & Training (Back-end)**
    - Programming language: **Python 3.10+**.
    - Framework: **PyTorch 2.x** for research and training of CNN/YOLO-based models.
    - Supporting libraries: torchvision, Albumentations (data augmentation), scikit-learn (metrics), Ultralytics YOLO for YOLOv8-style models.[6] [9] [2]
2. **Model Export & Optimization**
    - Intermediate format: **ONNX** for framework-agnostic export.
    - Inference/edge format: **TensorFlow Lite (TFLite)** as the primary optimized format for mobile and AR deployment.
    - Techniques: Post-training quantization (int8/float16) and pruning using TFLite tooling and/or PyTorch export pipelines.[8] [2] [3] [6]

3. **Mobile Application & Inference (Android)**
    - Programming language: **Kotlin** (primary), with **Java** only where necessary.
    - ML runtime: **TensorFlow Lite Android** with delegates (GPU/NNAPI).
    - Platform: **Android Studio** toolchain and SDK.[10] [2] [3]
4. **Augmented Reality Glasses & AR Integration**
    - AR framework on Android: **ARCore** (or OpenXR-based SDK if required by specific AR hardware).
    - 3D/AR experience: **Unity** as the 3D engine when OEM recommends it; integrate TFLite via Unity plugins.
    - For see-through AR devices that expose Android APIs, reuse the Kotlin + TFLite stack.[10]
5. **Computer Vision & Pre/Post-processing**
    - Image/video processing: **OpenCV** (Python for training, native/Android ports for deployment).
    - Optional: **MediaPipe** for high-level real-time CV graphs (e.g., camera input, tracking) if integration effort is acceptable.[2] [10]

These technologies form the reference architecture for training, optimizing, and deploying AI models on mobile devices and AR glasses for rice crop health monitoring.[3] [2]

3. Technical Criteria

The selection is justified against the following criteria:

1. **Computational Efficiency & Model Size**
    - Target models: MobileNetV2/V3, ShuffleNetV2, lightweight YOLO variants (e.g., YOLOv8-nano/YOLO-DP/SSD-YOLO), with parameters typically <10M and FLOPs well below 1–2 GFLOPs, enabling real-time inference on entry-level devices.[4] [7] [3] [5] [6] [8] [2]
    - Literature shows models like LiSA-MobileNetV2 and lightweight YOLO variants achieve >95% accuracy on rice disease detection while staying small enough for mobile deployment.[3] [5] [8] [2]
2. **Offline Operation**
    - All inference must run **on-device** without network, due to connectivity issues in rice-growing regions and privacy of captured images.[5] [2]
    - TFLite specifically targets on-device inference with optimized kernels and delegates for ARM CPUs and mobile GPUs.[10]
3. **Compatibility with Entry-level Mobile Devices**
    - Target hardware: 2–4 GB RAM, mid-range Snapdragon/MediaTek SoCs.
    - TFLite and MobileNet/YOLO-nano variants are widely validated as suitable for such devices in agricultural and general mobile CV tasks.[6] [2] [3] [5]

4. **Cross-platform & Ecosystem Support**
    - PyTorch is widely used in research and supports rapid experimentation with new architectures (e.g., improved YOLO, attention mechanisms).[9] [2] [6]
    - ONNX enables conversion to multiple runtimes; TFLite has mature Android integration and community examples for mobile/edge scenarios.[10]
5. **Maintainability and Team Skills**
    - Python for ML is standard in academia and industry; Kotlin is now the recommended language for Android apps.[10]
    - PyTorch + TFLite has extensive documentation and examples, reducing onboarding and maintenance cost.[2] [6] [10]

4. Restrictions and Constraints

1. **Hardware Constraints**
    - Limited RAM and compute on entry-level phones and AR glasses restrict models to light architectures and aggressive quantization.[5] [2]
    - AR devices may have thermal and battery limitations, limiting sustained high FPS and heavy GPU use.[10]
2. **Connectivity Constraints**
    - No reliable cloud connectivity; cloud inference (e.g., TensorFlow Serving, SageMaker endpoints) is excluded for real-time detection.[2]
3. **Regulatory and Data Constraints**
    - Farmer images may contain sensitive content; on-device processing simplifies privacy and compliance.
4. **Platform Constraints**
    - The main target platform is **Android** (phones and Android-based AR). iOS and Apple Vision Pro are out of scope for the first phase and therefore Core ML is not chosen as primary.[10]
5. **Model Family Constraints**
    - Heavy backbones (e.g., ResNet-50/101, standard YOLOv5-x) are excluded from deployment, even if used in early experiments or baselines, due to memory and latency constraints.[4] [6] [2]

5. Options Considered and Justification

5.1 Training Framework

Option A – PyTorch (Selected)

- Pros:
  - Strong research adoption, easy prototyping of new CNN and YOLO variants.[9] [6] [2]
  - Robust ecosystem (Ultralytics YOLO, Albumentations) for detection and segmentation.[6] [9]
  - Straightforward export to ONNX and then to TFLite.[10]
- Cons:
  - Requires extra steps for some quantization workflows vs. native TensorFlow.[10]
- Decision: Selected for flexibility and alignment with recent rice disease/pest detection work, which heavily uses PyTorch and YOLO toolchains.[8] [2] [6]

Option B – TensorFlow/Keras (Not Selected as Primary)

- Pros:
  - Native integration with TFLite and many mobile examples.[10]
- Cons:
  - Less convenient for YOLOv8-style experiments; part of the recent YOLO research uses PyTorch.[9] [6]
- Decision: May be used for specific experiments, but not the primary framework.

5.2 Inference Runtime

Option A – TensorFlow Lite (Selected)

- Pros:
  - Specifically optimized for mobile/edge, with delegates for NNAPI, GPU, and Hexagon DSP.[10]
  - Very common in mobile agriculture and plant disease detection use cases.[2] [5] [6]
  - Supports post-training quantization to shrink models and speed up inference.[3] [8] [6]
- Cons:
  - Extra conversion step from PyTorch via ONNX.[10]
- Decision: Selected as the main runtime for Android and AR devices.

Option B – ONNX Runtime (Not Selected as Primary)

- Pros:
  - Cross-platform (Android, iOS, desktop, edge).[10]
- Cons:
  - Android optimization and delegate ecosystem for entry-level hardware is less mature than TFLite.[10]

- Decision: Kept as a secondary option if a specific device does not support TFLite well.

5.3 Mobile Platform Stack

Option A – Kotlin + TFLite Android (Selected)

- Pros:
  - Kotlin is the official Android language, with modern language features and better tooling. [10]
  - TFLite has official Android libraries and examples in Kotlin. [10]
- Cons:
  - Android-specific; iOS support would require a separate codebase.
- Decision: Selected as aligned with the main deployment target and ecosystem.

Option B – Flutter or React Native + ONNX/TFLite (Not Selected as Primary)

- Pros:
  - Cross-platform UI.
- Cons:
  - More complex ML integration and often worse access to low-level acceleration APIs. [10]
- Decision: Not selected; may be revisited if multi-platform UI needs become dominant.

5.4 AR Platform Stack

Option A – ARCore + Unity + TFLite (Selected)

- Pros:
  - ARCore provides plane detection and tracking for overlaying diagnosis results on leaves. [10]
  - Unity is supported by several AR glasses vendors and can integrate TFLite via plugins.
- Cons:
  - Added complexity vs. pure Android apps; performance tuning needed on AR devices.
- Decision: Selected as a flexible approach across AR devices that expose Android/OpenXR support.

6. Consequences

Positive Consequences

- Consistent end-to-end pipeline (PyTorch → ONNX → TFLite → Android/AR) validated by multiple recent studies on lightweight rice disease and pest detection. [8] [3] [5] [6] [2]
- Strong support for offline, low-latency inference on entry-level devices, crucial for field deployment. [5] [2] [10]

- Ability to reuse the same models across smartphone apps and AR glasses, reducing duplication.[10]

Negative Consequences / Risks

- The Android- and TFLite-centric choice reduces immediate portability to iOS and some proprietary AR platforms, which could require additional conversion work (e.g., to Core ML).[10]
- Conversion and quantization steps can introduce accuracy degradation (e.g., a few percentage points loss when moving to int8).[3] [6] [8]
- AR performance (battery, thermals) must be monitored; the chosen stack does not automatically solve power constraints.[10]

7. References (Sources Used in the Decision)

- Surveys and reviews on crop and rice disease/pest detection with ML/CV:[1] [11] [12] [13] [2]
- Rice-specific lightweight models and YOLO variants (e.g., SSD-YOLO, LiSA-MobileNetV2, YOLO-DP, TLI-YOLO):[7] [4] [6] [8] [2] [3] [5]
- Guidance and templates for Architectural Decision Records:[14] [15] [16] [10]
- User/project context (efficient, offline, entry-level mobile deployment for rice detection):

⁂

1. interests.agriculture_tech.rice_disease_detection
2. https://pmc.ncbi.nlm.nih.gov/articles/PMC10088765/
3. https://pmc.ncbi.nlm.nih.gov/articles/PMC11253532/
4. https://pmc.ncbi.nlm.nih.gov/articles/PMC12370686/
5. https://ar5iv.labs.arxiv.org/html/1812.01043
6. https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2025.1643096/full
7. https://www.arxiv.org/pdf/2408.01752.pdf
8. https://www.techscience.com/cmc/v75n2/52135
9. https://pmc.ncbi.nlm.nih.gov/articles/PMC12528418/
10. https://arxiv.org/html/2501.06740v1
11. https://docs.cloud.google.com/architecture/architecture-decision-records
12. https://arccjournals.com/journal/indian-journal-of-agricultural-research/A-6061
13. https://www.rcs.cic.ipn.mx/2023_152_10/Pests Detection in Agricultural Crops Using Computer Vision.pdf
14. https://repositorio.iscte-iul.pt/bitstream/10071/26147/1/article_90301.pdf
15. https://github.com/joelparkerhenderson/architecture-decision-record
16. https://adr.github.io
17. https://adr.github.io/adr-templates/